



Universidad Nacional Autónoma de México
Facultad de Estudios Superiores Acatlán

Práctica

Práctica de concurrencia y paralelismo

Presenta

Jiménez Pineda Leydi Monserrat

N° de cuenta

421089037

Profesor

JOSÉ GUSTAVO FUENTES CABRERA

Materia

Programación Paralela y Concurrente

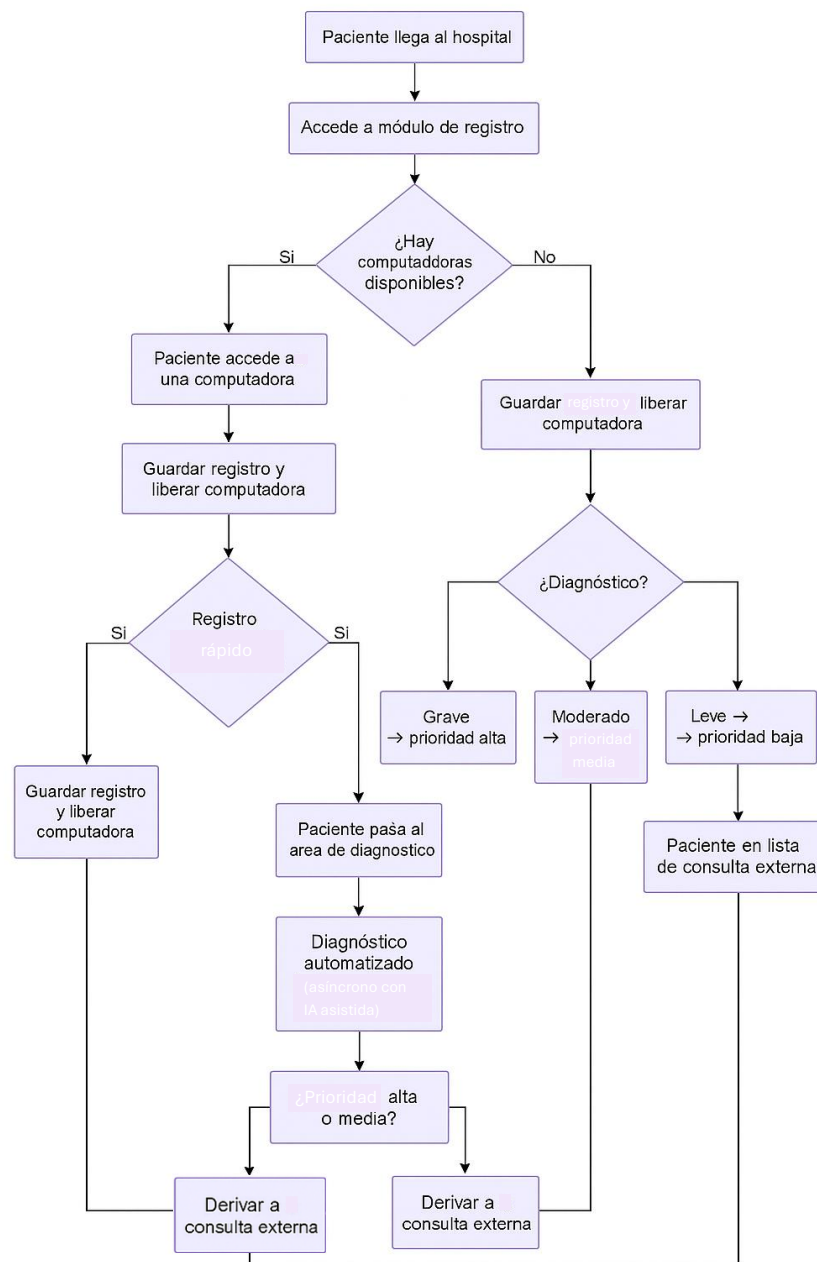
Santa Cruz Acatlán, Naucalpan, Estado de México



Objetivo

Aplicar y diferenciar los paradigmas de programación paralela, concurrente y asíncrona mediante la simulación de un sistema hospitalario automatizado. El sistema representa un entorno realista que requiere el procesamiento distribuido de tareas en distintos tiempos y recursos, abarcando etapas como el registro de pacientes, diagnóstico automático, asignación de recursos hospitalarios y alta médica.

Diagrama del sistema



Paciente

Para representar a cada individuo que participa en la simulación del sistema hospitalario, se definió una clase denominada Paciente. Esta estructura encapsula toda la información necesaria para el manejo del paciente a lo largo del flujo hospitalario. Los atributos incluidos son:

- ID: Identificador único generado automáticamente.
- Nombre: Nombre del paciente.
- Género: Masculino o femenino.
- Edad: Edad en años.
- Síntomas: Descripción breve de lo que presenta el paciente al llegar al hospital.
- Antecedentes (opcional): Lista de condiciones médicas previas.
- Prioridad: Asignada tras el diagnóstico (alta, media o baja).
- Diagnóstico: Resultado simulado asignado mediante lógica de IA.

Justificación del Uso de Cada Paradigma

El sistema implementa tres paradigmas de programación para simular de manera realista el entorno hospitalario:

1. Concurrencia (threading y Semaphore)

- **Aplicación:** Registro de pacientes y asignación de recursos.
- **Justificación:** Permite manejar múltiples pacientes que llegan al hospital simultáneamente, simulando estaciones de registro y recursos limitados como camas y doctores.

2. Asincronía (asyncio)

- **Aplicación:** Diagnóstico de pacientes.
- **Justificación:** Simula llamadas a servicios externos (como una IA médica) que pueden tener latencia, permitiendo que otros procesos continúen sin bloqueo.

3. Paralelismo (multiprocessing)

- **Aplicación:** Seguimiento y alta de pacientes.
- **Justificación:** Permite que múltiples pacientes sean atendidos en paralelo, simulando un entorno hospitalario donde varios pacientes están en seguimiento simultáneamente.

Explicación Detallada del Diseño

El diseño del sistema sigue un flujo lógico que refleja el proceso real en un hospital:

1. **Registro:** Los pacientes llegan y se registran. Si ya están en el sistema, el proceso es más rápido. Se utiliza concurrencia para manejar múltiples registros simultáneamente.
2. **Diagnóstico:** Cada paciente es diagnosticado de manera asincrónica, asignando una prioridad basada en sus síntomas.
3. **Asignación de Recursos:** Los pacientes con prioridad alta o media son asignados a camas y doctores disponibles, utilizando semáforos para controlar el acceso.
4. **Seguimiento y Alta:** Los pacientes son monitoreados y dados de alta en procesos paralelos, simulando un entorno hospitalario realista.

Fragmentos Clave de Código con Explicación

Este bloque modela de forma realista la **concurrencia en el área de registro** de un hospital, donde múltiples pacientes llegan al mismo tiempo, pero solo pueden ser atendidos si hay una computadora libre.

- `cola_pacientes`: es una cola segura para múltiples hilos donde se almacenan los pacientes ya registrados.
- `computadoras`: es un semáforo con capacidad limitada a 3, simulando que solo hay 3 computadoras disponibles para registrar pacientes al mismo tiempo.

La función `registrar_paciente` simula el registro de un paciente.

- El `with computadoras`: asegura que solo 3 hilos a la vez puedan entrar (por las 3 computadoras disponibles).
- `log_event("Registro-Inicio", paciente)` y `log_event("Registro-Fin", paciente)` registran los eventos de inicio y fin del proceso de registro.
- `time.sleep(...)` simula el tiempo que tarda el registro, aleatoriamente entre 0.5 y 1.0 segundos.
- El paciente se agrega a la cola `cola_pacientes` cuando finaliza su registro.

La función `iniciar_registro` lanza un hilo por cada paciente, para que todos puedan registrarse en paralelo.

- Se crea un hilo con `threading.Thread`, asignando como tarea la función `registrar_paciente`.
- Luego se inicia el hilo con `hilo.start()`.
- Finalmente, se espera que todos los hilos terminen con `hilo.join()`.

```
[4] cola_pacientes = queue.Queue()
    computadoras = threading.BoundedSemaphore(3)

    def registrar_paciente(paciente):
        with computadoras:
            log_event("Registro-Inicio", paciente)
            time.sleep(random.uniform(0.5, 1.0))
            cola_pacientes.put(paciente)
            log_event("Registro-Fin", paciente)

    def iniciar_registro(pacientes):
        hilos = []
        for paciente in pacientes:
            hilo = threading.Thread(target=registrar_paciente, args=(paciente,))
            hilos.append(hilo)
            hilo.start()
        for hilo in hilos:
            hilo.join()
```

Este bloque representa cómo un hospital podría diagnosticar a múltiples pacientes de manera simultánea, sin tener que esperar a que uno termine para comenzar con el siguiente, aprovechando la asincronía para no bloquear la ejecución del sistema.

Se define una función asincrónica, lo que significa que puede suspenderse (esperar) sin bloquear la ejecución de otras tareas. Se registra el inicio del diagnóstico. Luego se simula un tiempo de espera (como si se consultara una IA médica o API externa) que dura entre 1 y 2 segundos, de manera no bloqueante gracias a `await`.

- Se elige al azar un diagnóstico entre leve, moderado o grave.
- Según el diagnóstico, se asigna una prioridad de atención médica:
 - grave → prioridad alta
 - moderado → prioridad media
 - leve → prioridad baja

Se registra que el diagnóstico del paciente ha terminado. Se devuelve el objeto paciente con los atributos actualizados (diagnóstico y prioridad).

```
0 s [5] async def diagnosticar(paciente):
    log_event("Diagnóstico-Inicio", paciente)
    await asyncio.sleep(random.uniform(1.0, 2.0))
    diagnostico = random.choice(["leve", "moderado", "grave"])
    paciente.diagnostico = diagnostico
    if diagnostico == "grave":
        paciente.prioridad = "alta"
    elif diagnostico == "moderado":
        paciente.prioridad = "media"
    else:
        paciente.prioridad = "baja"
    log_event("Diagnóstico-Fin", paciente)
    return paciente

    async def diagnosticar_pacientes(lista_pacientes):
        tareas = [diagnosticar(p) for p in lista_pacientes]
        resultados = await asyncio.gather(*tareas)
        return resultados
```

Este código reproduce de manera realista el comportamiento de un hospital:

- Pacientes con urgencia baja no colapsan los recursos críticos.
- Los pacientes esperan si no hay camas o doctores disponibles, garantizando control de acceso simultáneo.
- El uso de semáforos permite manejar múltiples pacientes de forma concurrente, sin sobrecargar el sistema.

Si el paciente tiene prioridad baja, no se le asignan camas ni doctores. En su lugar, se deriva a consulta externa, y el proceso termina con return.

Se registra que el paciente está esperando recursos.

La línea with camas, doctores: bloquea el acceso hasta que haya una cama y un doctor disponibles.

Cuando ambos recursos están disponibles:

- Se simula el inicio de atención.
- Se espera entre 1 y 2 segundos para representar el tratamiento.
- Se finaliza la atención y los recursos se liberan automáticamente al salir del bloque with.

```

✓ 0 s [6] camas = threading.Semaphore(3)
doctores = threading.Semaphore(2)

def asignar_recursos(paciente):
    if paciente.prioridad == "baja":
        log_event("Derivado a consulta externa", paciente)
        return
    log_event("Espera de recursos", paciente)
    with camas, doctores:
        log_event("Atención-Inicio", paciente)
        time.sleep(random.uniform(1.0, 2.0))
        log_event("Atención-Fin", paciente)

```

Representa cómo un hospital sigue y da de alta a múltiples pacientes al mismo tiempo, como si estuvieran en diferentes salas o en manos de distintos equipos médicos.

A diferencia del registro (que usa hilos) y el diagnóstico (que es asincrónico), aquí se usa paralelismo real para evitar que el trabajo de un paciente bloquee a otro.

Esta función representa el proceso final de atención al paciente. Se imprime un mensaje indicando que el paciente está en seguimiento. Luego, se simula una espera entre 1 y 2 segundos (tratamiento u observación). Al finalizar, se indica que el paciente ha sido dado de alta.

Esta función lanza procesos en paralelo (no hilos) para los pacientes con prioridad alta o media. Process proviene de multiprocessing, y permite que cada paciente se maneje en un núcleo separado del CPU. Se inicia cada proceso con `p.start()` y luego se espera su finalización con `p.join()`.

```

✓ 0 s [7] def seguimiento(paciente):
    print(f"[Seguimiento] {paciente.nombre} en seguimiento intensivo...")
    time.sleep(random.uniform(1.0, 2.0))
    print(f"[Alta] {paciente.nombre} ha sido dado de alta.")

def iniciar_seguimiento(pacientes):
    procesos = []
    for paciente in pacientes:
        if paciente.prioridad in ["alta", "media"]:
            p = Process(target=seguimiento, args=(paciente,))
            procesos.append(p)
            p.start()
    for p in procesos:
        p.join()

```

Resultados de Pruebas y Rendimiento

La simulación se ejecutó en un entorno controlado con múltiples pacientes definidos en memoria. A continuación, se presentan los resultados observados en el log de ejecución, los cuales reflejan correctamente el comportamiento esperado del sistema bajo condiciones de concurrencia, asincronía y paralelismo.

Al ejecutar todo en Colab Google tuvimos los siguientes resultados

```
INICIO DEL FLUJO

[Registro-Inicio] (18:17:56) Ana (ID:8, Edad:30, Prioridad:media)
[Registro-Inicio] (18:17:56) Luis (ID:9, Edad:40, Prioridad:media)
[Registro-Inicio] (18:17:56) María (ID:10, Edad:25, Prioridad:media)
[Registro-Fin] (18:17:56) María (ID:10, Edad:25, Prioridad:media)
[Registro-Inicio] (18:17:56) Pedro (ID:11, Edad:50, Prioridad:media)
[Registro-Fin] (18:17:56) Luis (ID:9, Edad:40, Prioridad:media)
[Registro-Inicio] (18:17:56) Valeria (ID:12, Edad:60, Prioridad:media)
[Registro-Fin] (18:17:56) Ana (ID:8, Edad:30, Prioridad:media)
[Registro-Inicio] (18:17:56) José (ID:13, Edad:35, Prioridad:media)
[Registro-Fin] (18:17:57) Valeria (ID:12, Edad:60, Prioridad:media)
[Registro-Inicio] (18:17:57) Lucía (ID:14, Edad:28, Prioridad:media)
[Registro-Fin] (18:17:57) Pedro (ID:11, Edad:50, Prioridad:media)
[Registro-Fin] (18:17:57) José (ID:13, Edad:35, Prioridad:media)
[Registro-Fin] (18:17:58) Lucía (ID:14, Edad:28, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) Ana (ID:1, Edad:30, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) María (ID:3, Edad:25, Prioridad:alta)
[Diagnóstico-Inicio] (18:17:58) Luis (ID:2, Edad:40, Prioridad:baja)
[Diagnóstico-Inicio] (18:17:58) Valeria (ID:5, Edad:60, Prioridad:alta)
[Diagnóstico-Inicio] (18:17:58) Pedro (ID:4, Edad:50, Prioridad:baja)
[Diagnóstico-Inicio] (18:17:58) José (ID:6, Edad:35, Prioridad:alta)
[Diagnóstico-Inicio] (18:17:58) Lucía (ID:7, Edad:28, Prioridad:baja)
[Diagnóstico-Inicio] (18:17:58) María (ID:10, Edad:25, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) Luis (ID:9, Edad:40, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) Ana (ID:8, Edad:30, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) Valeria (ID:12, Edad:60, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) Pedro (ID:11, Edad:50, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) José (ID:13, Edad:35, Prioridad:media)
[Diagnóstico-Inicio] (18:17:58) Lucía (ID:14, Edad:28, Prioridad:media)
[Diagnóstico-Fin] (18:17:59) Ana (ID:8, Edad:30, Prioridad:alta)
[Diagnóstico-Fin] (18:17:59) Pedro (ID:4, Edad:50, Prioridad:baja)
[Diagnóstico-Fin] (18:17:59) José (ID:6, Edad:35, Prioridad:alta)
[Diagnóstico-Fin] (18:17:59) Valeria (ID:12, Edad:60, Prioridad:media)
[Diagnóstico-Fin] (18:17:59) Luis (ID:9, Edad:40, Prioridad:baja)
[Diagnóstico-Fin] (18:17:59) Valeria (ID:5, Edad:60, Prioridad:media)
[Diagnóstico-Fin] (18:17:59) Ana (ID:1, Edad:30, Prioridad:media)
```



```
[Diagnóstico-Fin] (18:17:59) Ana (ID:1, Edad:30, Prioridad:media)
[Diagnóstico-Fin] (18:18:00) María (ID:10, Edad:25, Prioridad:baja)
[Diagnóstico-Fin] (18:18:00) José (ID:13, Edad:35, Prioridad:baja)
[Diagnóstico-Fin] (18:18:00) Lucía (ID:14, Edad:28, Prioridad:baja)
[Diagnóstico-Fin] (18:18:00) Lucía (ID:7, Edad:28, Prioridad:media)
[Diagnóstico-Fin] (18:18:00) Luis (ID:2, Edad:40, Prioridad:baja)
[Diagnóstico-Fin] (18:18:00) María (ID:3, Edad:25, Prioridad:alta)
[Diagnóstico-Fin] (18:18:00) Pedro (ID:11, Edad:50, Prioridad:alta)
[Espera de recursos] (18:18:00) Ana (ID:1, Edad:30, Prioridad:media)
[Atención-Inicio] (18:18:00) Ana (ID:1, Edad:30, Prioridad:media)
[Espera de recursos] (18:18:00) María (ID:3, Edad:25, Prioridad:alta)
[Atención-Inicio] (18:18:00) María (ID:3, Edad:25, Prioridad:alta)
[Derivado a consulta externa] (18:18:00) Luis (ID:2, Edad:40, Prioridad:baja)
[Espera de recursos] (18:18:00) Valeria (ID:5, Edad:60, Prioridad:media)
[Derivado a consulta externa] (18:18:00) Pedro (ID:4, Edad:50, Prioridad:baja)
[Espera de recursos] (18:18:00) José (ID:6, Edad:35, Prioridad:alta)
[Espera de recursos] (18:18:00) Lucía (ID:7, Edad:28, Prioridad:media)
[Derivado a consulta externa] (18:18:00) María (ID:10, Edad:25, Prioridad:baja)
[Derivado a consulta externa] (18:18:00) Luis (ID:9, Edad:40, Prioridad:baja)
[Espera de recursos] (18:18:00) Ana (ID:8, Edad:30, Prioridad:alta)
[Espera de recursos] (18:18:00) Valeria (ID:12, Edad:60, Prioridad:media)
[Espera de recursos] (18:18:00) Pedro (ID:11, Edad:50, Prioridad:alta)
[Derivado a consulta externa] (18:18:00) José (ID:13, Edad:35, Prioridad:baja)
[Derivado a consulta externa] (18:18:00) Lucía (ID:14, Edad:28, Prioridad:baja)
[Atención-Fin] (18:18:01) Ana (ID:1, Edad:30, Prioridad:media)
[Atención-Inicio] (18:18:01) Valeria (ID:5, Edad:60, Prioridad:media)
[Atención-Fin] (18:18:01) María (ID:3, Edad:25, Prioridad:alta)
[Atención-Inicio] (18:18:01) Lucía (ID:7, Edad:28, Prioridad:media)
[Atención-Fin] (18:18:02) Lucía (ID:7, Edad:28, Prioridad:media)
[Atención-Inicio] (18:18:02) Ana (ID:8, Edad:30, Prioridad:alta)
[Atención-Fin] (18:18:03) Valeria (ID:5, Edad:60, Prioridad:media)
[Atención-Inicio] (18:18:03) Valeria (ID:12, Edad:60, Prioridad:media)
[Atención-Fin] (18:18:03) Ana (ID:8, Edad:30, Prioridad:alta)
[Atención-Inicio] (18:18:03) José (ID:6, Edad:35, Prioridad:alta)
[Atención-Fin] (18:18:04) Valeria (ID:12, Edad:60, Prioridad:media)
[Atención-Inicio] (18:18:04) Pedro (ID:11, Edad:50, Prioridad:alta)
[Atención-Fin] (18:18:05) José (ID:6, Edad:35, Prioridad:alta)
```

```
[Atención-Fin] (18:18:05) José (ID:6, Edad:35, Prioridad:alta)
[Atención-Fin] (18:18:06) Pedro (ID:11, Edad:50, Prioridad:alta)
[Seguimiento] Ana en seguimiento intensivo...
[Seguimiento] María en seguimiento intensivo...
[Seguimiento] Valeria en seguimiento intensivo...[Seguimiento] José en seguimiento intensivo...

[Seguimiento] Lucía en seguimiento intensivo...
[Seguimiento] Ana en seguimiento intensivo...
[Seguimiento] Valeria en seguimiento intensivo...
[Seguimiento] Pedro en seguimiento intensivo...
[Alta] María ha sido dado de alta.
[Alta] Ana ha sido dado de alta.
[Alta] Pedro ha sido dado de alta.
[Alta] José ha sido dado de alta.
[Alta] Valeria ha sido dado de alta.
[Alta] Lucía ha sido dado de alta.
[Alta] Ana ha sido dado de alta.
[Alta] Valeria ha sido dado de alta.
```

1. Registro concurrente:

- El sistema registró a múltiples pacientes al mismo tiempo, respetando la restricción de 3 computadoras disponibles usando BoundedSemaphore.
- Todos los pacientes fueron correctamente agregados a la cola de espera tras un tiempo de atención variable (0.5 a 1.0 s).
- El proceso de registro se completó rápidamente y sin bloqueos.

2. Diagnóstico asincrónico:

- Todos los pacientes fueron diagnosticados de forma simultánea mediante `asyncio.gather()`.
- Se asignó correctamente una prioridad (alta, media, baja) basada en el diagnóstico aleatorio.
- El diagnóstico no bloqueó al resto del sistema gracias a su naturaleza asincrónica.

3. Asignación de recursos concurrente:

- Los pacientes con prioridad **baja** fueron derivados automáticamente a consulta externa, liberando recursos.
- Los pacientes con prioridad alta o media ingresaron al área médica, esperando recursos controlados con semáforos:
 - 3 camas disponibles
 - 2 doctores disponibles
- El acceso fue secuenciado y gestionado correctamente: ningún paciente ingresó sin recursos disponibles.

4. Seguimiento en paralelo:

- Pacientes atendidos pasaron a seguimiento usando `multiprocessing.Process`.
- Todos los procesos corrieron en paralelo, simulando observación médica intensiva.
- Cada paciente fue dado de alta después de un tratamiento aleatorio (1 a 2 segundos).

Uso de inteligencia Artificial

Durante el desarrollo de esta práctica, se utilizó inteligencia artificial (IA) a través de la herramienta ChatGPT como asistente técnico, con el objetivo de facilitar y enriquecer el proceso de diseño y codificación del sistema hospitalario simulado. Fue apoyo para el diseño del flujo hospitalario, propuso ideas para estructurar el recorrido de un paciente desde el registro hasta el alta médica. Sugirió una estructura escalable para el proyecto, alineada con buenas prácticas de ingeniería de software.

Explicó y aplicó el uso correcto de:

- `threading` y `BoundedSemaphore` para concurrencia.
- `asyncio` para diagnóstico asincrónico.
- `multiprocessing` para seguimiento en paralelo.

La IA ofreció ejemplos, comparaciones y fragmentos funcionales adaptados al entorno de Google Colab.

Repositorio

Colab

<https://colab.research.google.com/drive/1oTe9c5NP1DQZ1DFnZY78ROwXsg0875ZX?usp=sharing>

Git hub

https://github.com/LeydiMoon18/hospital_simulacion#